Application Versioning Policy

Application Versioning Policy

Purpose

This document defines a simple, consistent, and customer-friendly versioning policy for all application releases. The goal is to make versions meaningful to customers, easy to manage for developers, and reliable for support and audit purposes.

---

1. Version Format

We use **Semantic Versioning (SemVer)** in the following format:

```xml
MAJOR.MINOR.PATCH
Example: 1.4.2

```

Each part has a specific meaning and must be incremented intentionally.

---

2. What Each Number Means

PATCH (1.4.0 → 1.4.1)

Used for bug fixes and internal improvements

Increment **PATCH** when:

- Bug fixes
- Performance improvements
- Security fixes
- UI / layout corrections
- Validation or error-handling fixes
- Logging or monitoring changes

✅ Existing functionality remains unchanged

✅ No customer retraining required

✅ No breaking changes

MINOR (1.4.0 → 1.5.0)

Used for backward-compatible feature additions

Increment **MINOR** when:

- New feature or module is added
- New screen, report, or API endpoint
- Optional configuration or setting added
- Enhancements that customers can choose to use

✅ Existing features continue to work

✅ No mandatory changes for customers

MAJOR (1.4.0 → 2.0.0)

Used for breaking changes

Increment **MAJOR** when:

- Existing APIs change or are removed
- Database schema changes are not backward compatible
- Features are removed or behavior is fundamentally changed
- Configuration changes are mandatory
- Upgrade requires explicit customer action or retraining

⚠️ Major releases must be communicated clearly to customers

⚠️ Upgrade instructions and backups are required

---

3. What Versions Should NOT Be Used For

Do **not** increment versions for:

- Every code commit
- Every test build
- Local development builds
- CI or pipeline executions

Version numbers are **customer-facing**, not developer counters.

---

4. Build Identification (Technical Traceability)

Each release ZIP includes additional identifiers:

```xml
<app>-<version>-<git-commit>-<timestamp>.zip
```

Example:

```xml
```

```
sfl_startup-1.0.1-7f79f90-20260130-1857.zip
```

This ensures:

- Unique builds
- Full traceability
- Easy rollback and audit

✅ These identifiers do **not** replace the version number.

---

5. Version Management Rules

- `<Version>` or `<VersionPrefix>` is defined in the `.csproj`
- Version is updated **only when delivering a new build to the customer**
- Multiple builds of the same version are allowed
- Version must be incremented **before** packaging a new customer delivery

---

6. Pre-Release Versions (Optional)

For UAT, pilot, or validation builds:

XML

```
1.2.0-beta.1
1.2.0-rc.1
```

Pre-release versions:

- Are not production releases
- Must not replace stable versions

---

7. Quick Decision Guide

Ask one question:

> **Does the customer need to change anything to use this release?**

	Situation	Increment
1	Bug fix only	PATCH

2	New optional feature	MINOR
3	Breaking or mandatory change	MAJOR

---

8. Responsibility

- Developers follow this policy when preparing releases
- Release owner approves version changes
- Support and QA reference versions exactly as delivered

Summary Rule

> *Versions describe customer impact. Build details describe technical traceability.*

This policy applies to all current and future application projects.